TRIP GENI — SYSTEM BLUEPRINT & LEARNING GUIDE

Complete Technical Architecture, Frontend-Backend Connections, Core Concepts, and Function Encyclopedia in Simple Words

Project: Trip Geni (AI Travel Platform)	Target Audience: Full-Stack Engineers & Interview Prep
Primary LLM Engine: OpenAI gpt-4o (Dual-Fallback Gemini)	Frontend: Next.js 15.5 App Router + React 19 + TypeScript
Backend: FastAPI + SQLAlchemy + SQLite (WAL)	Live Hosting: Vercel (Frontend) + Render (Backend API)
Core Capabilities: Tourism RAG, Token Bucket Rate Limiting, TQI Evaluation, Rail/Flight Live Pricing	Document Version: 2.0 (November 2026 Production Edition)

1. Executive Summary & Problem Statement

What is Trip Geni? Trip Geni is a production-grade, AI-driven sustainable travel planning and intelligence platform. Unlike generic travel aggregators (e.g. MakeMyTrip, Expedia) that present disconnected static hotel/flight search boxes, and unlike vanilla ChatGPT/LLM chat boxes that hallucinate non-existent trains, unrealistic distances, and fake budgets, Trip Geni acts as an end-to-end intelligent travel architect. It synthesizes personalized, mathematically verified day-by-day itineraries, computes real-world multi-modal transit costs (flights, IRCTC trains, highway fuel, tolls), predicts live regional weather, measures eco-travel carbon footprints, and continuously learns from post-trip expense feedback.

The Core Problems Trip Geni Solves:
• The Hallucination Problem in Travel AI: LLMs routinely invent trains that do not run, schedule sightseeing at 2 AM, or assume travel between cities is instant. Trip Geni eliminates this by anchoring all LLM generation inside a grounded Tourism RAG (Retrieval-Augmented Generation) pipeline.
• The Realistic Budget Problem: Travelers struggle to estimate actual costs. Trip Geni allocates budgets mathematically across Accommodation, Inter-City Transit, Daily Local Meals, and Activities, backed by a Q-learning budget calibration engine.
• The Multi-Modal Transit Gap: Travelers don't know whether flying, taking a Vande Bharat train, or driving is most cost- and time-effective. Trip Geni computes multi-modal comparisons using live fuel benchmarks and official APIs.

CORE VALUE PROPOSITION

Trip Geni combines generative AI intelligence with deterministic engineering safeguards: verified regional data, token bucket rate limiters, multi-rubric evaluation scoring, and offline fallbacks to guarantee 100% platform availability.

2. End-to-End System Architecture & Data Flow

Trip Geni is designed as a decoupled, multi-tier cloud application separating presentation (Next.js 15 on Vercel) from business logic, data modeling, and AI orchestration (FastAPI on Render).

Layer	Technologies	Primary Responsibilities
1. Presentation Layer (Client)	Next.js 15.5 (App Router) React 19, TypeScript Tailwind CSS, Leaflet 1.9 Lucide Icons, Web Speech API	Renders responsive travel wizards, interactive geospatial Leaflet route maps, dynamic world clocks, mascot companion dialog, token bucket popup alerts, and client-side 0ms cache state.
2. API Gateway & Middleware (Server Edge)	FastAPI (Python 3.13) Starlette BaseHTTPMiddleware Token Bucket Rate Limiter CORSMiddleware	Enforces endpoint-specific token bucket quotas (Auth vs Booking vs General), sanitizes request payloads with Pydantic v2 schemas, handles JWT authorization, and manages CORS headers.

3. Intelligence & RAG Layer (AI Engine)	OpenAI API (gpt-4o / gpt-4o-mini) Google Gemini (2.5 / 1.5 Flash) TourismRAGService EvaluationService (TQI)	Grounded retrieval over 700+ regional destinations. Primary synthesis via OpenAI gpt-4o, automated failover to Gemini, and deterministic offline synthesis. Computes multi-rubric RAG evaluation metrics.
4. External Services & APIs (Data Ingestion)	OpenWeatherMap API CARTO Voyager Basemaps OpenStreetMap Nominatim API Mitra (IRCTC & Fuel) Ignav Flight Fares	Provides live temperature & weather forecasts, geospatial coordinate resolution, official Indian Railways train schedules, real-time petrol/diesel rates, and verified flight booking links.
5. Persistence & Analytics (Data Store)	SQLite 3 (WAL Mode) SQLAlchemy 2.0 ORM Auto-Migration Engine Budget RL Service	Stores user profiles, credentials (bcrypt hashes), saved trips, expense histories, and learned budget variance weights. High concurrency PRAGMAs ensure lock-free reads and fast writes.

3. Frontend Architecture & Page Inventory

The frontend is built using Next.js 15.5 App Router with full TypeScript strict typing and React 19. It employs hybrid server/client rendering: structural layouts load instantly from the server, while interactive dashboards, planners, and modals use 'use client'; to manage client-side state, event emitters, and persistent local storage caching.

Route	Key Responsibilities & Components	Engineering Highlights
/ (Home)	Brand landing page, dynamic hero section with smart scroll cues, platform statistics badge, floating Mascot preview, and instant navigation search.	Optimized image loading, CSS glassmorphism, responsive navigation drawer, and sub-second First Contentful Paint.
/planner (Trip Wizard)	4-Step Intelligent Trip Planner: Step 1 Destination & Multi-City; Step 2 Dates & Travelers; Step 3 Travel Style & Transit; Step 4 Budget Allocation & Focus Points.	Dynamic date range calculation (gradual to-date auto-update when entering day counts), custom traveler count pills (max 5 + manual input), dynamic tier slider (Saver, Comfort, Bespoke).
/trip/generated (Itinerary Dashboard)	The core generated trip view: Day-by-Day schedule tabs, sequential route map, recommended stays with Google ratings, restaurant highlights, weather alerts, and PDF download button.	Integrates TripInteractiveMap with Leaflet 1.9, numbered markers, live itinerary budget donut chart, and quantitative TQI evaluation score badge.
/dashboard (Travel Hub)	Personalized traveler cockpit: recent trips, quick weather search widget, carbon footprint overview, and the Smart World Clock & Travel Compass utility.	Real-time local clock + personalized saved world clocks with live second ticking; device magnetometer orientation compass integration.
/profile (User Account)	Traveler identity card, Eco-Travel Score (0-100), total trips taken, saved itineraries list, and Edit Profile modal.	0ms Cold Start Hydration: Instantly renders cached user state from localStorage; uses 3.5s AbortController timeout so slow backend wakeups never freeze the screen.
/ai-chat (Talk with Tripzy)	Conversational AI travel assistant in Tripzy's warm, enthusiastic voice. Supports destination queries, packing suggestions, budget breakdowns, and transit links.	Multi-turn conversation history tracking, Web Speech API speech-to-text input and voice readout, and automatic multi-language support (English, Manglish, Malayalam, Tamil).
/login & /register (Auth Pages)	Glassmorphism authentication forms with show/hide password toggles, field validation, and social login redirects.	Guarded by client-side token bucket rate limiter; parses FastAPI Pydantic 422 error arrays into friendly human sentences; handles case-insensitive emails and usernames.

4. Backend Architecture & Service Catalog

The backend is developed with FastAPI running on Python 3.13, utilizing async ASGI request handling via Uvicorn. It features strict separation of concerns into Controllers (API Routers), Business Logic (Services), Data Persistence (SQLAlchemy Models), and Validation Contracts (Pydantic Schemas).

Module Path	Core Purpose	Key Functions & Classes
app/core/rate_limiter.py	Thread-safe Token Bucket rate limiting engine with automatic stale client eviction.	TokenBucket, TokenBucketManager, RateLimitMiddleware, classify_endpoint(), BUCKET_CONFIGS

app/services/ai_service.py	Master AI orchestration service managing OpenAI gpt-4o, Gemini fallback, and offline synthesis.	generate_trip_plan(), _generate_content(), generate_chat_response(), generate_offline_trip_plan(), calculate_sustainability_score()
app/services/tourism_rag_service.py	Domain-specific regional Tourism Retrieval-Augmented Generation (RAG) knowledge engine.	TourismRAGService, retrieve_context_for_query(), resolve_destination(), resolve_route_pair(), detect_language()
app/services/evaluation_service.py	Quantitative evaluation framework computing the Trip Quality Index (TQI) across 5 rubrics.	EvaluationService, evaluate_itinerary(), _calculate_faithfulness(), _calculate_budget_adherence(), run_benchmark_suite()
app/services/budget_learning_service.py	Reinforcement Learning / Q-learning inspired budget feedback and calibration loop.	BudgetLearningService, record_expense_feedback(), get_destination_insights(), get_learned_category_weight()
app/api/transport.py	Multi-modal transportation intelligence endpoints (Trains, Flights, Fuel, Road trip).	search_trains(), search_flights_get(), get_fuel_prices(), estimate_road_trip_cost(), compare_modes()
app/database/db.py	Database engine, connection pooling, SQLite WAL settings, and auto-migration engine.	create_database() (with dynamic SQLite ALTER TABLE column migrations), get_db() dependency generator, SessionLocal

5. The Connection: Frontend-to-Backend Lifecycle

Understanding how the Next.js frontend connects to the FastAPI backend is vital for full-stack comprehension and technical interviews.

1. Dynamic Base URL Resolution (apiConfig.ts):
In production on Vercel, the browser cannot connect to localhost:8000. The frontend uses getApiBaseUrl() which inspects process.env.NEXT_PUBLIC_API_URL. In development, it defaults to http://127.0.0.1:8000. In production, it routes to the live Render backend (https://tripgeni.onrender.com).

2. Axios Request & Response Interceptors:
• Request Interceptor: Intercepts all outgoing HTTP calls in authService and tripService. Automatically extracts the JWT Bearer token from localStorage.getItem('tripgenius_token') and injects the header: Authorization: Bearer <token>.
• Response Interceptor (Rate Limit Catcher): Inspects incoming HTTP responses. If a status code of 429 Too Many Requests is detected, it reads the Retry-After header, triggers the global event triggerRateLimitAlert(), and opens the RateLimitAlertModal with a real-time countdown.
• 422 Validation Error Parser: If FastAPI returns HTTP 422 Unprocessable Entity, the interceptor parses Pydantic's error array into plain English sentences instead of failing with [object Object].

3. Cross-Origin Resource Sharing (CORS):
The backend implements Starlette's CORSMiddleware. Requests from Vercel deployment domains (e.g. https://tripgenius-kohl.vercel.app or any preview URL matching ^https://.*\.vercel\.app$) are recognized and granted cross-origin access with allow_credentials=True.

ARCHITECTURAL INSIGHT

Because CORSMiddleware is wrapped as the outermost middleware, even HTTP 429 Too Many Requests and HTTP 422 Unprocessable Entity responses carry proper CORS headers, preventing opaque browser network errors.

6. AI Intelligence, OpenAI gpt-4o & Tourism RAG Pipeline

Current AI Runtime Reality: 100% OpenAI gpt-4o
In your active backend/.env, OPENAI_API_KEY is set to an active key with OPENAI_MODEL=gpt-4o, while GEMINI_API_KEY is unconfigured. In ai_service.py, OpenAI is configured as Priority 1. Therefore, all live generative AI tasks (itinerary synthesis, budget narrative, and companion chat) execute through OpenAI gpt-4o.

The Dual-Model Fallback Hierarchy:
1. Priority 1 (OpenAI gpt-4o): First attempts generation via OpenAI with strict JSON mode (response_format={'type': 'json_object'}) and 30s timeout.
2. Priority 2 (Google Gemini 2.5 Flash): If OpenAI credentials are absent or fail, automatically fails over to Gemini.
3. Priority 3 (Grounded Offline RAG Synthesis): If both external APIs are unreachable (or network times out), the local deterministic Tourism RAG engine synthesizes a verified, rich multi-day plan with 0 downtime.

How Tourism RAG Prevents Hallucinations:
Before sending any user prompt to OpenAI, TourismRAGService.retrieve_context_for_query() searches a curated regional database containing verified facts for over 700 destinations and major transit corridors. It retrieves factual data: real attractions, local culinary dishes, authentic hotel pricing brackets, and verified IRCTC railway transit times. These facts are prepended into the OpenAI prompt as VERIFIED REGIONAL RAG FACTS with the strict system instruction: 'Adhere strictly to verified facts. Do not invent fictitious places or impossible transit routes.'

7. Token Bucket Rate Limiting Architecture

What is the Token Bucket Algorithm?
Imagine a bucket with a fixed capacity of tokens (e.g. 5 tokens). Every second, new tokens drip into the bucket at a constant refill rate (e.g. 1 token every 12 seconds). When a user makes a request, 1 token is removed from the bucket. If the bucket has tokens, the request is allowed immediately (burst capacity). If the bucket is empty, the request is rejected with HTTP 429 Too Many Requests and informed exactly how many seconds to wait until a token refills.

Bucket Category	Target Endpoints	Capacity	Refill Rate	Cooldown & Purpose
Auth Bucket	/api/auth/login /api/auth/register /api/auth/send-password-otp	5 tokens	5 / min (1 per 12s)	Strict burst control. Guards against brute-force password guessing and account spamming.
Booking Bucket	/api/trips/create /api/trips/generate-ai-itinerary /api/transport/flights/booking-links /api/transport/trains/*	10 tokens	10 / min (1 per 6s)	Prevents spamming expensive OpenAI itinerary generation and external rail/flight APIs.
General Bucket	All other /api/* routes (weather, profile, history)	60 tokens	60 / min (1 per 1s)	Permits smooth, fast browsing and dashboard navigation without throttling.

Endpoint Isolation Guarantee: The buckets are completely segregated by client IP. If a user exhausts their login attempts (Auth Bucket), they are NOT locked out of browsing flights or saving trips (Booking Bucket).

8. Quantitative Evaluation Metrics Framework (Trip Quality Index / TQI)

To evaluate RAG generation quality quantitatively, Trip Geni implements a formal 5-rubric evaluation engine in evaluation_service.py. Every generated trip receives a mathematical Trip Quality Index (TQI, 0-100%) and letter grade (A+, A, B+, B, C, D).

Rubric Pillar	Weight	Mathematical Measurement & Logic
1. RAG Faithfulness & Groundedness	30%	Cross-checks generated attractions, stays, and food against grounded RAG knowledge. Penalizes unverified entities.
2. Budget Adherence	25%	Evaluates financial precision: max(0, 1 - |estimated - budget| / budget) across lodging, transit, and food.
3. Pacing & Schedule Feasibility	20%	Checks temporal feasibility: 2-4 activities per day, non-overlapping morning/afternoon/evening slots, and travel buffers.
4. Context & Interest Relevance	15%	Measures the presence of user interests (nature, food, heritage, adventure) in selected venues and day themes.
5. Eco-Sustainability Rating	10%	Scores multi-modal transit carbon impact and eco-conscious habits (rail vs air, shared transit, local dining).

9. Key Technical Terms Explained in Simple Words

Here is a plain-English translation of the advanced software engineering and AI terms used in Trip Geni:

• RAG (Retrieval-Augmented Generation): Giving an AI an open-book exam. Instead of asking the AI to guess from memory, we first look up verified facts in our tourism database and give those facts to the AI so it writes an accurate answer.

• Token Bucket Rate Limiting: A fair queueing system that lets users perform normal quick actions (bursts), but prevents bots or spam scripts from overwhelming the server by requiring a token for every request.

• JWT (JSON Web Token): A secure digital badge given to the user after logging in. The frontend saves this badge and presents it in the header of every request so the server knows who is calling without re-checking passwords.

• WAL Mode (Write-Ahead Logging): An advanced SQLite database optimization. In standard mode, only one person can touch the database at a time. In WAL mode, many users can read data at the same time while someone else is writing.

• Cold Start (Render 15-min Sleep): Free cloud servers (like Render) go to sleep after 15 minutes of inactivity to save electricity. When a new user arrives, the server takes 30-50 seconds to boot up. This is a cloud infrastructure behavior, NOT an application rate limit or bug.

• Pydantic Validation (HTTP 422): A strict bouncer for incoming data. If an API expects an email and a password, and the user sends numbers or missing fields, Pydantic immediately rejects the request with HTTP 422 before bad data touches the database.

• Geocoding (Nominatim 1 RPS): Converting a city or attraction name into GPS coordinates (Latitude/Longitude). OpenStreetMap's Nominatim allows only 1 request per second (1 RPS). Trip Geni prevents IP bans by queuing lookups with 800ms delays and local caching.

• Q-Learning / Budget RL: A machine learning technique where the system learns from actual user trip receipts. If travelers consistently spend 10% more on food in Munnar than expected, the algorithm automatically adjusts future budget estimates.

10. Critical Functions Catalog & Plain English Explanations

Below are the most important functions across the codebase, where they live, what they do, and how to explain them in an interview:

Function & Location	What It Does (Simple Words)	Interview Explanation Tip
consume() rate_limiter.py	Checks if a client has enough tokens left in their bucket to perform an action. If yes, subtracts 1 token and returns allowed=True. If no, calculates the exact seconds until the next token drops.	'I implemented a thread-safe token bucket using time.monotonic() to eliminate clock drift and accurately enforce category quotas.'
RateLimitMiddleware.dispatch() rate_limiter.py	Intercepts every incoming HTTP request. Classifies the route into Auth, Booking, or General, checks the caller's IP bucket, and either returns HTTP 429 with Retry-After or passes the request to the route.	'Placed behind CORS so error responses retain required headers. Prevents CORS browser blockage during 429 exhaustion.'
_generate_content() ai_service.py	The core AI communication pipeline. Sends the master itinerary prompt to OpenAI gpt-4o in JSON mode. If OpenAI fails, falls back to Gemini. If Gemini fails, triggers deterministic offline generation.	'Architected with a dual-model fallback hierarchy to achieve 99.9% itinerary synthesis availability.'
retrieve_context_for_query() tourism_rag_service.py	Analyzes user destination and route queries, searches the 700+ destination dataset, and returns verified regional facts (highlights, IRCTC trains, road distance, authentic food) to inject into the LLM prompt.	'This is our RAG retriever. It grounds the LLM in regional facts to eliminate hallucinations.'
evaluate_itinerary() evaluation_service.py	Takes any generated plan and runs multi-rubric evaluation: Faithfulness, Budget Adherence, Pacing, Context Relevance, and Eco Score, returning the composite Trip Quality Index (TQI).	'Demonstrates quantitative AI evaluation. We don't just generate text; we evaluate accuracy mathematically.'
create_database() db.py	Initializes SQLite database tables and inspects table PRAGMAs to automatically run ALTER TABLE migrations for newly added model columns (username, date_of_birth, etc.).	'Solves SQLite's limitation where create_all() ignores new columns on existing database files.'
getApiBaseUrl() apiConfig.ts (Frontend)	Inspects environment variables and resolves the backend URL. Points to Render in production and localhost:8000 in local development.	'Eliminates hardcoded hostnames, ensuring zero broken API connections between Vercel and Render.'

11. Deployment, DevOps & Production Resilience

Frontend Deployment (Vercel):
• Git push to main triggers automated Vercel CI/CD builds.
• All 23 Next.js routes are statically optimized (SSG) with client hydration where required.
• Turbopack enables instant hot module reloading during development.

Backend Deployment (Render):
• Runs as a Python 3.13 web service on Render, managed by Uvicorn ASGI on port 8000.
• Handling Render's 15-Minute Inactivity Sleep: Free tier containers spin down after 15 minutes of idle time. The first request takes 30-50 seconds to wake up.
• Our Engineering Solution:
1. Instant Cache Render: The profile and dashboard render locally cached state from localStorage immediately (0ms).
2. AbortController Timeouts: Added 3.5s timeout abort signals to profile lookups so the UI never freezes indefinitely.
3. Helpful Error Messages: Axios interceptor detects timeout and explains to the user: 'The travel server is waking up from idle sleep (Render cold start). Please wait a few seconds...'

12. Top 10 Technical Interview Questions & Model Answers

Use these structured answers when explaining Trip Geni in software engineering and AI internship interviews:

Q1: Walk me through the architecture of Trip Geni.

Trip Geni is a full-stack AI travel platform. The frontend is built on Next.js 15 App Router and React 19 deployed on Vercel. The backend is an asynchronous FastAPI service deployed on Render. Our AI engine uses OpenAI gpt-4o as the primary generator, grounded by a regional Tourism RAG service to prevent hallucinations. We use SQLite in WAL mode with auto-migrations, an endpoint-wise Token Bucket rate limiter, and a quantitative Trip Quality Index evaluation framework.

Q2: Why did you migrate from Gemini to OpenAI?

We originally prototyped with Gemini 1.5/2.5 Flash, but migrated to OpenAI gpt-4o for our primary pipeline because of its superior adherence to strict JSON schemas and more nuanced multi-turn conversational memory. However, we kept Gemini as an automated secondary fallback in code, giving us multi-model redundancy so our platform remains online even if one provider encounters an outage.

Q3: How does your Tourism RAG pipeline prevent hallucinations?

Vanilla LLMs frequently hallucinate closed attractions or non-existent direct transit. Our RAG pipeline intercepts user queries, searches our curated database of 700+ destinations and inter-city transit corridors, and extracts verified facts (geodesic distance, highway travel hours, IRCTC train numbers, hotel price brackets). We inject these verified facts into the prompt before calling OpenAI, forcing the model to ground its response in real data.

Q4: How does your Token Bucket rate limiter work?

We implemented a thread-safe Token Bucket algorithm in FastAPI middleware. Instead of a single blanket limit, we separated buckets by endpoint category: Auth has 5 tokens (refills every 12s) to prevent credential stuffing; Booking has 10 tokens (refills every 6s) to protect expensive AI and booking APIs; and General routes have 60 tokens. If exhausted, the backend returns HTTP 429 with a Retry-After header, and our frontend displays a glassmorphism countdown alert.

Q5: How do you evaluate the quality of your generated itineraries?

We implemented a quantitative evaluation service inspired by RAGAS. It scores each itinerary across 5 weighted pillars: RAG Faithfulness (30%), Budget Adherence (25%), Pacing & Schedule Feasibility (20%), Context Relevance (15%), and Eco-Sustainability (10%). This yields a composite Trip Quality Index (TQI) from 0 to 100% and a letter grade, which we also test across automated destination benchmarks.

Q6: How do you handle database migrations with SQLite?

In SQLite, standard SQLAlchemy create_all() will not add new columns to an existing table. In create_database(), we implemented an automatic column introspection engine using PRAGMA table_info. It detects missing columns at application startup and dynamically executes ALTER TABLE statements, ensuring smooth schema evolution without manual database wipes.

Q7: How did you solve the Render cold start issue?

On Render's free tier, the backend sleeps after 15 minutes of inactivity, causing a 30-50 second boot delay. To prevent the frontend from freezing on a white screen, we implemented optimistic local storage hydration (0ms instant render for profiles), paired with 3.5-second AbortController timeouts and friendly user notifications explaining the cloud wake-up cycle.

Q8: How do you handle OpenStreetMap Nominatim rate limits?

Nominatim strictly enforces a 1 request per second (1 RPS) policy. In our Leaflet map component, we implemented an 800ms staggered queue for sequential geocoding requests and cached resolved coordinate pairs in localStorage, completely eliminating 429 IP bans from OpenStreetMap.

Q9: What is the Eco-Travel Score and how is it calculated?

The Eco-Travel Score is an objective sustainability rating from 0 to 100. It evaluates multi-modal transit choices (rail and shared bus award points, while flights carry carbon penalties), eco-conscious activities (nature walks, wildlife sanctuaries), and green accommodation habits, encouraging lower-impact tourism.

Q10: What was the most challenging bug you resolved in this project?

One challenging issue was an HTTP 422 Unprocessable Entity error on login. It occurred because the login Pydantic schema was enforcing a minimum password length of 8 characters, rejecting shorter legacy credentials before database verification. We refactored the login schema to accept general strings while retaining strict validation on registration, implemented case-insensitive email matching, and improved error parsing so users receive actionable feedback.

Document End • Trip Geni Technical Handbook

Created for educational review, code mastery, and engineering excellence. All rights reserved.